

Competitive Programming

Section 54 Editorial: Geometry

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Orientation	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
3	B. Segment Intersection	5
3.1	Restatement	5
3.2	Observations and algorithm	5
3.3	Correctness proof	5
3.4	Complexity	5
3.5	Full C++ solution	5
3.6	Full Python solution	5
4	C. Polygon Double Area	7
4.1	Restatement	7
4.2	Observations and algorithm	7
4.3	Correctness proof	7
4.4	Complexity	7
4.5	Full C++ solution	7
4.6	Full Python solution	7
5	D. Interior Lattice Points	8
5.1	Restatement and theorem	8
5.2	Boundary count and worked trace	8
5.3	Algorithm	8
5.4	Correctness proof	9
5.5	Complexity, implementation notes, and tests	9
5.6	Full C++ solution	9
5.7	Full Python solution	10
6	What to carry forward	11
7	Complete upsolve notes	12
7.1	A. Orientation	12
7.2	B. Segment Intersection	12
7.3	C. Polygon Double Area	12

1 Editorial overview

1.1 Problem-specific proof map

A. Orientation. The cross product sign is positive, negative, or zero according to whether the turn from AB to AC is left, right, or collinear.

B. Segment Intersection. Proper crossings have opposite orientation signs; collinear boundary cases reduce to bounding-box overlap. These cases are exhaustive for closed segments.

C. Polygon Double Area. The shoelace sum adds signed trapezoid areas around the boundary. Interior contributions cancel, leaving twice the polygon area.

D. Interior Lattice Points. Pick's theorem relates area, boundary lattice points, and interior lattice points. An edge contributes $\gcd(|dx|, |dy|)$ boundary steps when summed cyclically, so no endpoint needs special treatment.

1.2 Detailed solution guide

A. Orientation. For points a, b, c , compute the cross product $(b-a) \times (c-a)$. Positive means c is to the left of directed line $a \rightarrow b$, negative means right, and zero means collinear. This sign is the signed doubled area of triangle abc , so it is robust with integer coordinates.

B. Segment Intersection. Two segments intersect properly when the endpoints of each segment lie on opposite sides of the other segment. Collinear cases are handled by checking whether an endpoint lies inside the bounding box of the other segment. Combining orientation signs with `on_segment` covers crossing, touching endpoints, and overlapping collinear segments.

C. Polygon Area. The shoelace formula sums $x_i y_{i+1} - y_i x_{i+1}$ around the polygon. This is the sum of signed triangle areas against the origin. The absolute value divided by two is the area; many contest problems print doubled area to avoid fractions.

Common pitfalls. Use 64-bit integers for cross products. Include the edge from the last vertex back to the first. For intersections, collinear cases are not optional.

All three geometry tasks are solved with integer cross products.

- A computes the sign of `cross(AB, AC)` to classify orientation.
- B combines orientation tests with bounding-box checks for collinear endpoint and overlap cases.
- C uses the shoelace sum and prints twice the polygon area.

No floating point is needed. The main implementation risk is overflow, so C++ solutions use `long long` for coordinate products.

2 A. Orientation

2.1 Restatement

Classify point C relative to directed segment AB.

2.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

2.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

2.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int T;if(!(cin>
```

2.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    t=data[0]; idx=1; out=[]
    for _ in range(t):
        ax,ay,bx,by,cx,cy=data[idx:idx+6]; idx+=6; v=(bx-ax)*(cy-ay)-(by-ay)*(cx-ax); out.ap
        print("\n".join(out))
if __name__=="__main__": main()
```

3 B. Segment Intersection

3.1 Restatement

Decide whether two closed segments intersect.

3.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

3.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

3.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; using P=pair<long long,long long>; long long ori(P a,P b,P c){return (b[0]-a[0])*(c[1]-a[1])-(b[1]-a[1])*(c[0]-a[0])}
```

3.6 Full Python solution

```
import sys
def ori(a,b,c): return (b[0]-a[0])*(c[1]-a[1])-(b[1]-a[1])*(c[0]-a[0])
def between(a,b,c): return min(a[0],b[0])<=c[0]<=max(a[0],b[0]) and min(a[1],b[1])<=c[1]<=max(a[1],b[1])
def inter(a,b,c,d):
    o1,o2,o3,o4=ori(a,b,c),ori(a,b,d),ori(c,d,a),ori(c,d,b)
    if o1==0 and between(a,b,c): return True
    if o2==0 and between(a,b,d): return True
    if o3==0 and between(c,d,a): return True
    if o4==0 and between(c,d,b): return True
```

```

    if o4==0 and between(c,d,b): return True
    return (o1>0)!=(o2>0) and (o3>0)!=(o4>0)
def main():
    data=list(map(int,sys.stdin.buffer.read().split())); t=data[0]; idx=1; out=[]
    for _ in range(t):
        x=data[idx:idx+8]; idx+=8; out.append("YES" if inter((x[0],x[1]),(x[2],x[3]),(x[4],x[5]),(x[6],x[7])) else "NO")
    print("\n".join(out))
if __name__=="__main__": main()

```

4 C. Polygon Double Area

4.1 Restatement

Print twice the area of a polygon in vertex order.

4.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

4.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

4.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

4.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; pts=[(data[i],data[i+1]) for i in range(1,2*n+1,2)]; s=0
    for i in range(n):
        x1,y1=pts[i]; x2,y2=pts[(i+1)%n]; s+=x1*y2-y1*x2
    print(abs(s))
if __name__=="__main__": main()
```

5 D. Interior Lattice Points

5.1 Restatement and theorem

Given a simple polygon whose vertices have integer coordinates, count lattice points strictly inside it. Vertices are in boundary order; consecutive collinear vertices are permitted. Coordinates reach 10^{12} , so both cross products and the total area require `__int128` in C++.

Pick's theorem for a simple lattice polygon states:

```
area = interior + boundary/2 - 1
2 * interior = double_area - boundary + 2
```

Thus we need exact double area from the shoelace formula and the number of lattice points on the boundary.

5.2 Boundary count and worked trace

For an edge with displacement (dx, dy) , the number of lattice *steps* along that edge is $\gcd(|dx|, |dy|)$. When these values are summed over all cyclic edges, every polygon vertex is accounted for exactly once, so:

```
boundary = sum gcd(|x[i+1]-x[i]|, |y[i+1]-y[i]|)
```

For the square $(0,0), (2,0), (2,2), (0,2)$, double area is 8. Each edge has gcd 2, so $\text{boundary}=8$. Pick gives:

```
interior = (8 - 8 + 2) / 2 = 1
```

The unique interior lattice point is $(1,1)$.

5.3 Algorithm

```
double_area_signed = 0
boundary = 0
for every cyclic edge P -> Q:
    double_area_signed += P.x * Q.y - P.y * Q.x
    boundary += gcd(abs(Q.x-P.x), abs(Q.y-P.y))
double_area = abs(double_area_signed)
print (double_area - boundary + 2) / 2
```

Use the final edge from the last vertex back to the first. The numerator is always nonnegative and even for a valid simple lattice polygon by Pick's theorem.

5.4 Correctness proof

The shoelace formula computes the polygon's exact doubled area from its ordered boundary. For each edge, lattice points on the segment occur at equally spaced steps of the primitive direction vector; their number of steps is the gcd of the coordinate differences. Summing gcd values cyclically counts every boundary lattice point exactly once, including vertices shared by edges.

Therefore the algorithm obtains the exact `double_area` and `boundary` of the given lattice polygon. Pick's theorem applies because the polygon is simple and all its vertices are lattice points. Rearranging it gives exactly $(\text{double_area} - \text{boundary} + 2)/2$, so the printed value is the number of strictly interior lattice points. QED.

5.5 Complexity, implementation notes, and tests

The algorithm is $O(n)$ time and $O(n)$ memory if vertices are stored (or $O(1)$ additional memory with streaming). In C++, cast before multiplying and accumulate in `__int128`; a `long long` product overflows for coordinates near 10^{12} . Python's integers are arbitrary precision. Test a unit-area triangle with zero interior points, clockwise vertex order, edges with gcd greater than one, and a polygon with collinear boundary vertices.

5.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Point {
    long long x, y;
};

void print_int128(__int128 value) {
    if (value == 0) {
        cout << 0;
        return;
    }
    string digits;
    while (value > 0) {
        digits.push_back(char('0' + value % 10));
        value /= 10;
    }
    reverse(digits.begin(), digits.end());
    cout << digits;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int n;
if (!(cin >> n)) return 0;
vector<Point> polygon(n);
for (Point& point : polygon) cin >> point.x >> point.y;

__int128 signed_double_area = 0;
__int128 boundary = 0;
for (int index = 0; index < n; index++) {
    Point a = polygon[index];
    Point b = polygon[(index + 1) % n];
    signed_double_area += (__int128)a.x * b.y - (__int128)a.y * b.x;
    boundary += gcd(llabs(b.x - a.x), llabs(b.y - a.y));
}
__int128 double_area = signed_double_area >= 0 ? signed_double_area : -signed_double_area;
__int128 interior = (double_area - boundary + 2) / 2;
print_int128(interior);
cout << '\n';
return 0;
}

```

5.7 Full Python solution

```

import math
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    polygon = [(data[index], data[index + 1]) for index in range(1, 2 * n + 1, 2)]

    signed_double_area = 0
    boundary = 0
    for index, (x1, y1) in enumerate(polygon):
        x2, y2 = polygon[(index + 1) % n]
        signed_double_area += x1 * y2 - y1 * x2
        boundary += math.gcd(abs(x2 - x1), abs(y2 - y1))
    interior = (abs(signed_double_area) - boundary + 2) // 2
    print(interior)

if __name__ == "__main__":
    main()

```

6 What to carry forward

7 Complete upsolve notes

7.1 A. Orientation

Compute cross $(bx-ax)*(cy-ay)-(by-ay)*(cx-ax)$. Its sign classifies left, right, or collinear because it is signed parallelogram area. $O(1)$; use long long and test all three signs and large coordinates.

7.2 B. Segment Intersection

For closed segments, proper crossing has opposite orientation signs on both segments; collinear endpoints require inclusive bounding-box `on_segment`. These cases are complete for two line segments. $O(1)$; test a shared endpoint, collinear overlap, collinear gap, and one segment containing another.

7.3 C. Polygon Double Area

Sum cross products of consecutive vertices including last-to-first, then take absolute value. Triangulation from the origin proves this signed shoelace sum. $O(n)$; test clockwise order, a triangle, and a rectangle. Python is exact; avoid floating point in C++ too.

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.

For D, tiny tests enumerate lattice points in the polygon's bounding box and use an exact ray test, independently of Pick's theorem. The scale case has 200,000 boundary vertices and coordinates of size 10^{12} .